Evaluación de Unidad

1. Datos Generales

Asignatura	Álgebra Lineal
Ciclo	1 A
Unidad	1
Resultado de aprendizaje de la unidad	Aplica y resuelve analíticamente utilizando diversos métodos sistemas de ecuaciones lineales, matrices y determinantes, aplicaciones lineales según diversos criterios y utilizando diversas técnicas para la resolución de problemas con ayuda de software matemático, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Nombre del Docente	Edwin René Guamán Quinche
Nombre del Estudiante	Daniela Loreley Alejandro Lozano
Fecha	04 al 10 de noviembre de 2026
Código en Github	https://github.com/daniela-aleja/AlgebraLineal/blob/main/Unidad1/cifrado_de_hill.ipynb

2. Objetivo:

- Demostrar la capacidad de identificar, modelar matemáticamente y resolver un problema práctico utilizando el álgebra lineal aplicando sistemas de ecuaciones lineales o matrices.

3. Contexto

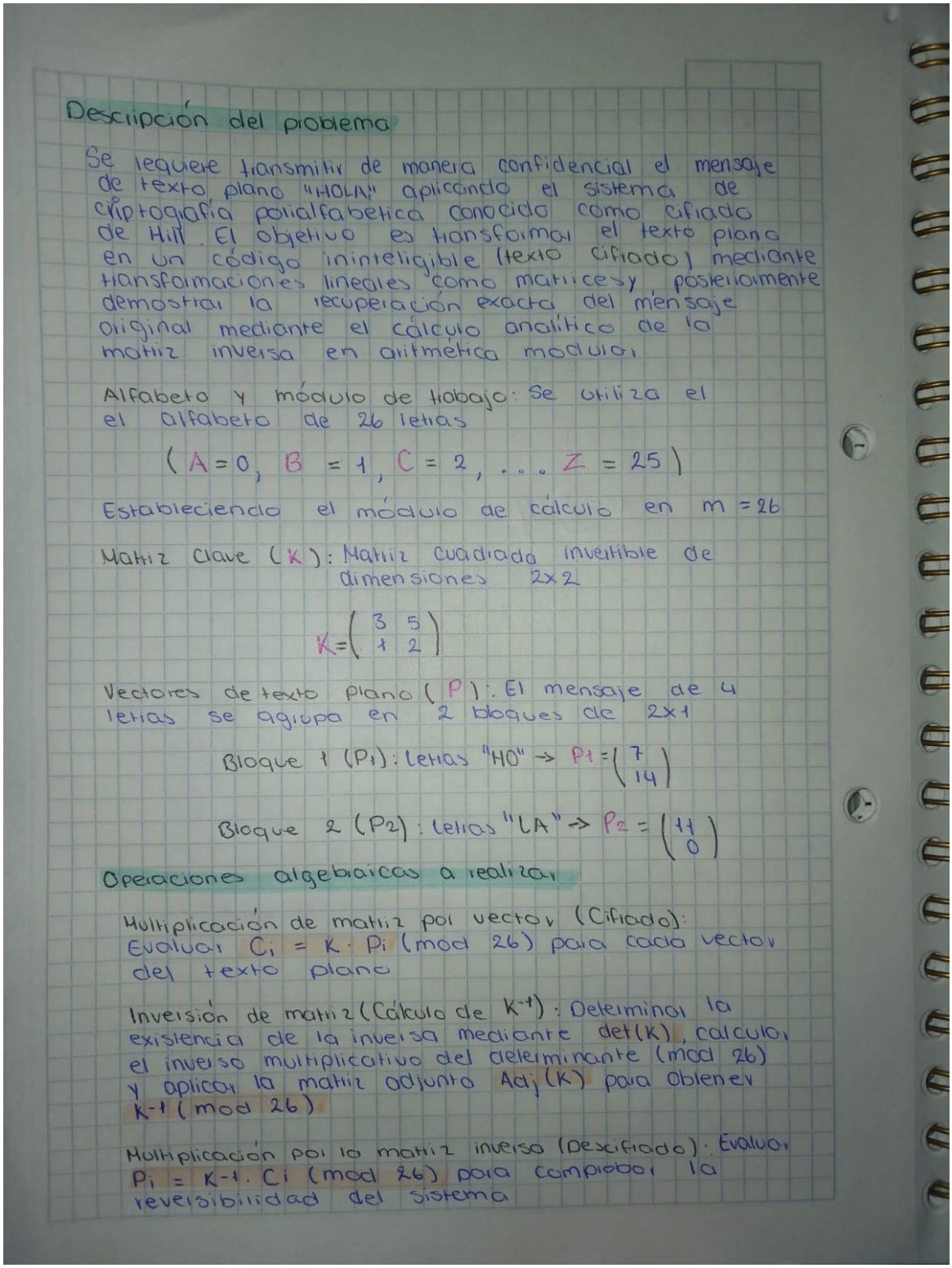
En este proyecto se estudia el cifrado de Hill. Este es un cifrado de sustitución polialfabético que utiliza la teoría de álgebra lineal. Es importante destacar que en el área de ingeniería de la computación uno de los aspectos más importantes es la seguridad y confidencialidad de la información que se envía por medio de las redes de computadoras.

Lo más interesante de este proyecto es que se muestra cómo se puede cifrar un texto plano utilizando operaciones matriciales básicas (multiplicación de matrices y cálculo de inversa por medio de determinantes) para obtener un texto cifrado e ilegible. Por otro lado, también se mostrará cómo se puede descifrar un texto ilegible para volverlo inteligible nuevamente.

El objetivo de este trabajo es entender cómo se pueden aplicar los conceptos de espacios matriciales y aritmética modular en la elaboración de algoritmos de seguridad informática. Además de esto se trabaja en el desarrollo de modelos de problemas reales por medio de código en Python y cálculos manuales analíticos.

4. Desarrollo de las operaciones

4.1. Planteamiento del problema



Descripción del problema

Se requiere transmitir de manera confidencial el mensaje de texto plano "HOLA" aplicando el sistema de criptografía polialfabetica conocido como cifrado de Hill. El objetivo es transformar el texto plano en un código ininteligible (texto cifrado) mediante transformaciones lineales como matrices y posteriormente demostrar la recuperación exacta del mensaje original mediante el cálculo analítico de la matriz inversa en aritmética modular

Alfabeto y módulo de trabajo: Se utiliza el el alfabeto de 26 letras

$(A=0,\ B=1,\ C=2,\ \ldots\ Z=25)$

Estableciendo el módulo de cálculo en $m=26$

Matriz clave (K): Matriz cuadrada invertible de dimensiones 2x2

$$K=\begin{pmatrix} 3 & 5 \\ 1 & 2 \end{pmatrix}$$

Vectores de texto plano (P): El mensaje de 4 letras se agrupa en 2 bloques de 2x1

Bloque 1 (P_1): Letras "HO" → $P_1=\begin{pmatrix} 7 \\ 14 \end{pmatrix}$

Bloque 2 (P_2): Letras "LA" → $P_2=\begin{pmatrix} 11 \\ 0 \end{pmatrix}$

Operaciones algebraicas a realizar

Multiplicación de matriz por vector (Cifrado): Evaluar $C_i = K \cdot P_i \pmod{26}$ para cada vector del texto plano

Inversión de matriz (Cálculo de K^{-1}): Determinar la existencia de la inversa mediante $\det(K)$, calcular el inverso multiplicativo del determinante (mod 26) y aplicar la matriz adjunta $Adj(K)$ para obtener $K^{-1} \pmod{26}$

Multiplicación por la matriz inversa (Descifrado): Evaluar $P_i = K^{-1} \cdot C_i \pmod{26}$ para comprobar la reversibilidad del sistema

Fig. 1. Planteamiento del cifrado de Hill para el mensaje "HOLA" con matriz clave $K \in \mathbb{Z}_{26}^{2x2}$.

4.2. Procedimiento paso a paso

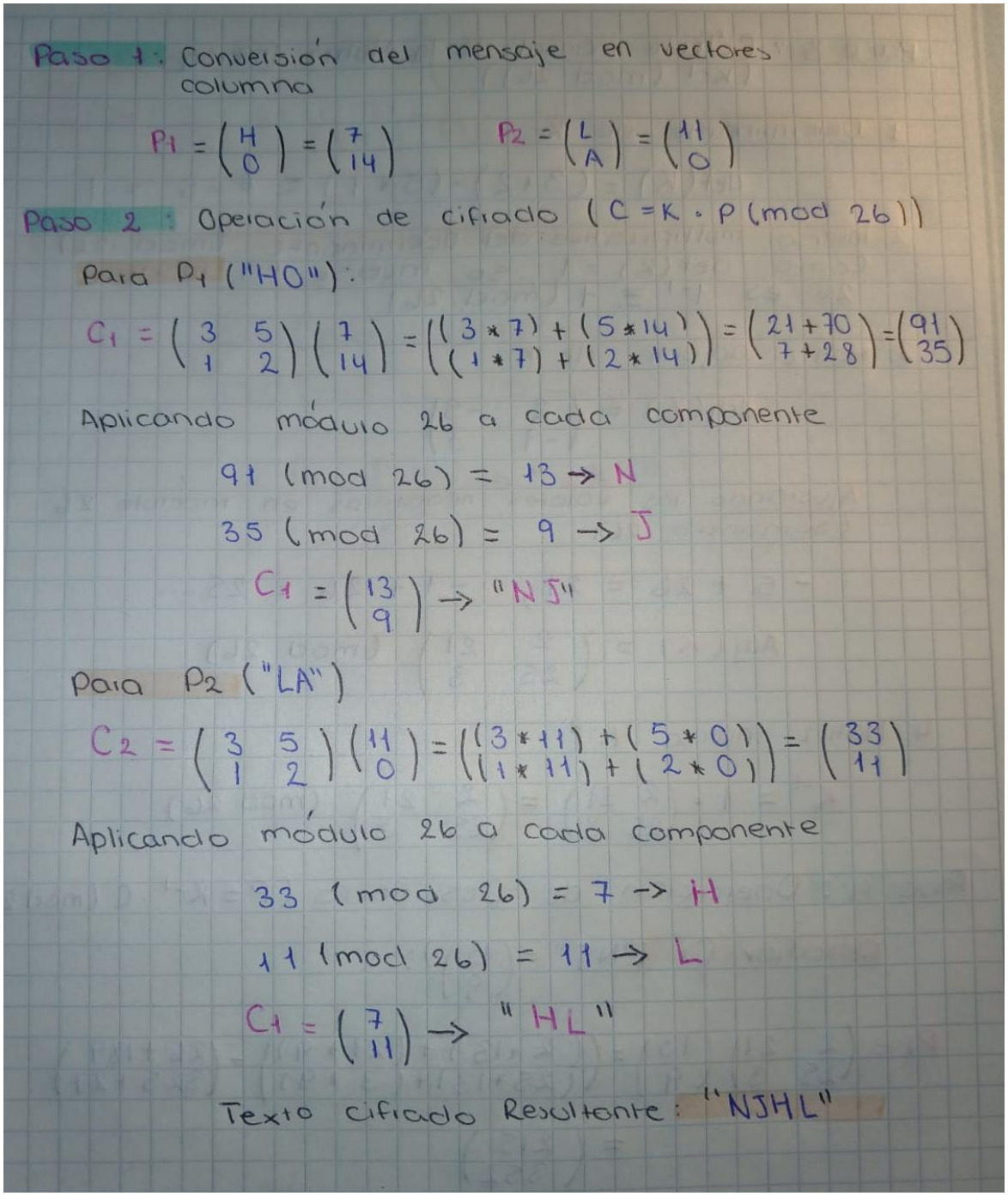


Fig. 2. *Cifrado del mensaje "HOLA" mediante C = K·P (mod 26): conversión a vectores columna, multiplicación matricial y reducción módulo 26, con texto cifrado resultante "NJHL".*

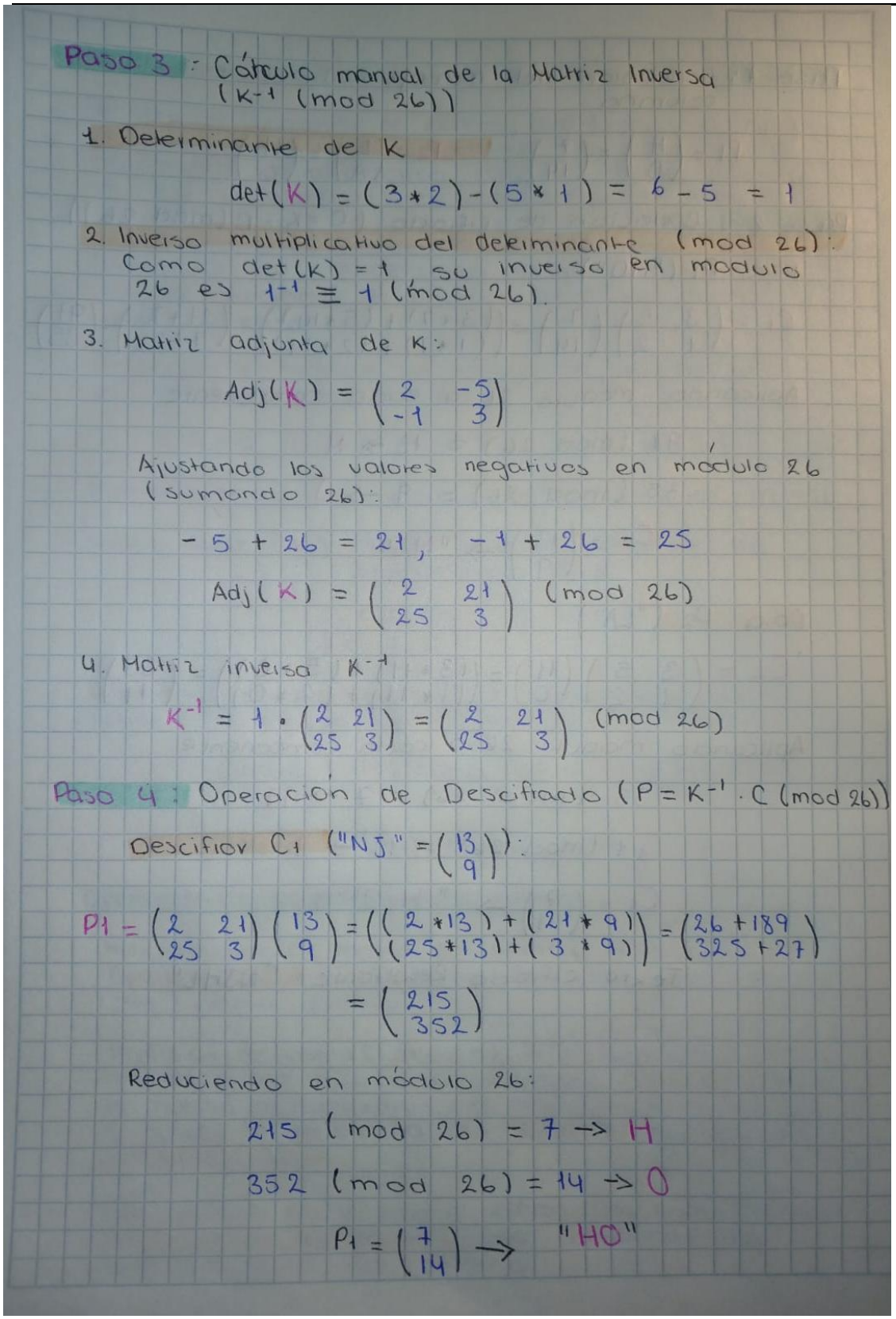


Fig. 3. *Cálculo manual de la matriz inversa K^{-1} (mod 26) mediante el determinante y la matriz adjunta, y descifrado del primer bloque C_1 = "NJ" para recuperar P_1 = "HO".*

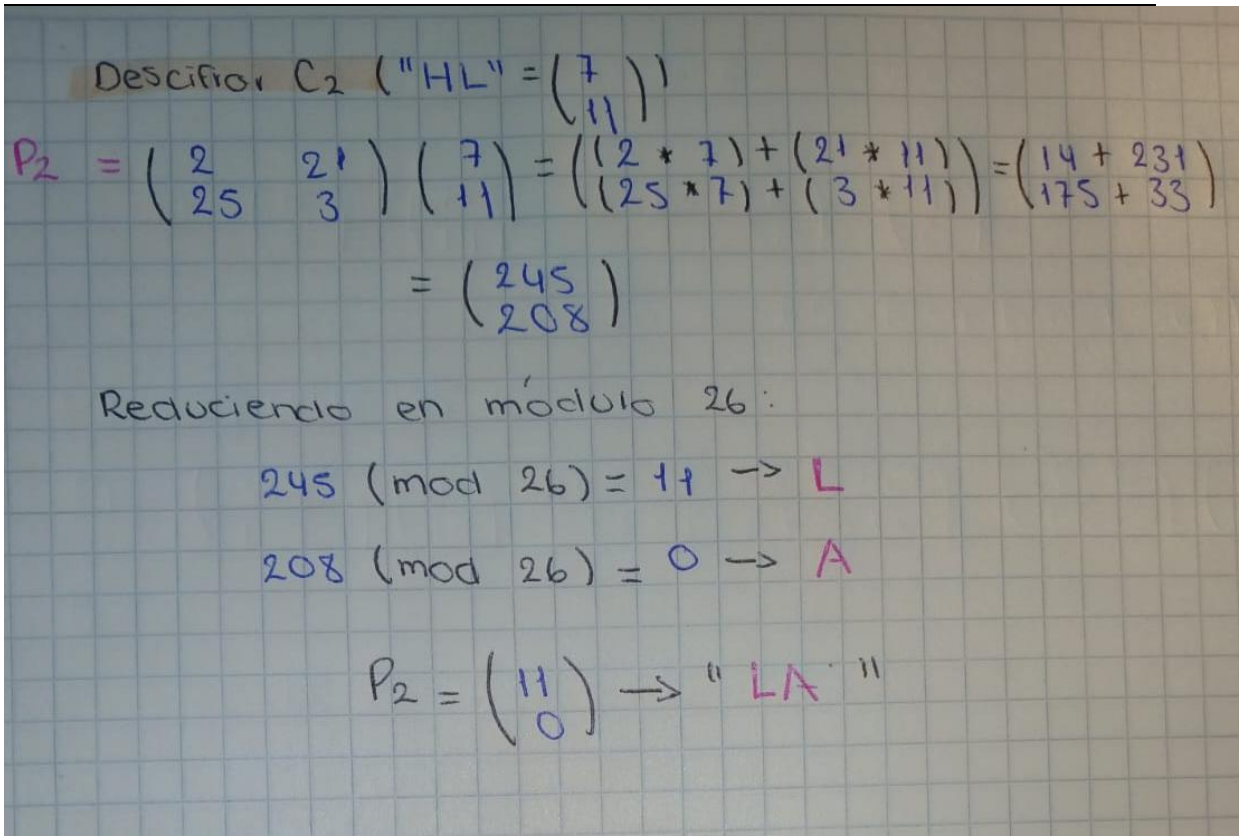


Fig. 4. *Descifrado del segundo bloque C_2 = "HL" mediante $P_2 = K^{-1} \cdot C_2$ (mod 26), obteniendo P_2 = "LA" y completando la recuperación del mensaje original "HOLA".*

4.3 Verificación de resultados

Al multiplicar la matriz inversa $\boldsymbol{K^{-1}}$ por los vectores cifrados C1 y C2, obtenemos nuevamente los vectores numéricos del texto plano original.

Para el bloque 1:

P1 = $\boldsymbol{K^{-1}} * \boldsymbol{C1}$ (mod 26) = $\begin{pmatrix}\mathbf{7}\\\mathbf{14}\end{pmatrix}$ -> (7 = H, 14 = O)

Para el bloque 2:

P2 = $\boldsymbol{K^{-1}} * \boldsymbol{C2}$ (mod 26) = $\begin{pmatrix}\mathbf{11}\\\mathbf{0}\end{pmatrix}$ -> (11 = L, 0 = A)

Al unir ambos bloques recuperados se obtiene la secuencia "HOLA", lo que confirma la validez algebraica de la transformación lineal y la efectividad del método

5. Procedimiento / Metodología

5.1 Descripción de la aplicación

En este proyecto se desarrolló un programa en Python (Google Colab) que implementa el cifrado de Hill para bloques de dimensión 2x2.

Está dirigida a estudiantes y profesionales de la ingeniería en computación interesados en la aplicación práctica del álgebra lineal en la criptografía y la seguridad de la información

Su objetivo principal es demostrar como las operaciones lineales con matrices (multiplicación de matrices por vectores y cálculo de la matriz inversa) garantizan la confidencialidad y la correcta recuperación de un mensaje

El programa modela el texto plano como una secuencia de vectores 2x1 y aplica una transformación lineal dada por la multiplicación C = K * P (mod 26). Para revertir el proceso, calcula analíticamente la matriz inversa K^{-1} mediante la matriz adjunta y el inverso multiplicativo del determinante en modulo 26

5.2 Descripción de la aplicación

Categoría	Herramienta / Tecnología	Versión / Detalle
Lenguaje de programación	Python	3.15.1
Entorno de desarrollo (IDE)	Google Colab	Entorno ejecutable en la nube
Librerías / Frameworks	NumPy	Manipulación de arreglos y calculo matricial

Plataforma de ejecución	Web / Nube	Ejecución multiplataforma en navegador

5.3 Funcionalidades implementadas

- **Funcionalidad 1: Configuración de la matriz clave K y vectorización del mensaje:** Permite definir la matriz K de 2x2 y convierte las letras en un vector numérico (A = 0, B = 1, …, Z =25)— **Aplica**: Representación de datos mediante vectores columna y matrices.
- **Funcionalidad 2**: **Cálculo de la matriz inversa modular (calcular_matriz_inversa_mod26):** Obtiene el determinante de la matriz, evalúa si es invertible mod 26 y calcula la adjunta para hallar $K^{-1}\ (mod\ 26)$— **Aplica**: Determinantes, matriz adjunta, inverso multiplicativo modular e inversión matricial
- **Funcionalidad 3: Módulo de cifrado (cifrar_hill):** Divide el mensaje en pares de letras y realiza el producto matricial entre la matriz clave K y cada vector P — **Aplica**: Multiplicación de matriz por vector (C = K * P (mod 26))
- **Funcionalidad 4: Módulo de descifrado (descifrar_hill):** Multiplica la matriz inversa K^{-1} por cada vector cifrado C para recuperar el texto original)— **Aplica:** Propiedad de la matriz inversa $(P = K^{-1} * C\ (mod\ 26))$

5.4 Capturas de pantalla

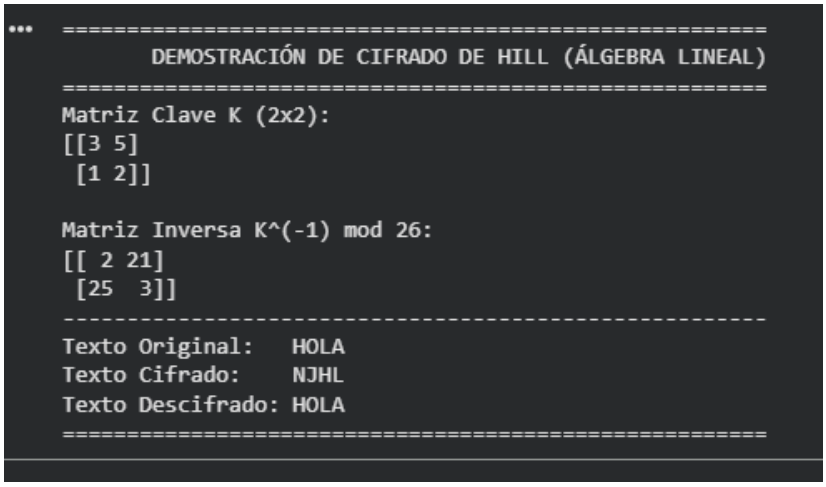


Fig. 5. Salida del programa en Python (Google Colab) que implementa el cifrado de Hill, texto original "HOLA", texto cifrado "NJHL" y texto descifrado "HOLA".

5.5 Código fuente (fragmentos clave)

Fragmento 1: Cálculo de la Matriz Inversa en Módulo 26.

```
def calcular_matriz_inversa_mod26(matriz):
     det = int(np.round(np.linalg.det(matriz)))
     det_mod = det % 26
     inv_det = inverso_modular(det_mod, 26) # Inverso del determinante mod
     26
     # Matriz Adjunta para 2x2: [[d, -b], [-c, a]]
     adj = np.array([
          [matriz[9], -matriz[9]],
          [-matriz[9], matriz] ])
     ])
     # K^(-1) = inv_det * Adj(K) mod 26
     matriz_inversa = (inv_det * adj) % 26
     return matriz_inversa.astype(int)
```

Fragmento 2: Multiplicación de matriz por vector para cifrar

```
def cifrar_hill(texto, clave):
     texto = texto.upper().replace(" ", "")
     if len(texto) % 2 != 0:
     texto += "X"  # Relleno si la cantidad de letras es impar
     texto_cifrado = ""
     for i in range(0, len(texto), 2):
          v = np.array([[ord(texto[i]) - 65], [ord(texto[i+1]) - 65]])
          res = np.dot(clave, v) % 26
          texto_cifrado += chr(res[0, 0] + 65) + chr(res[1, 0] + 65)

     return texto_cifrado
```

Repositorio / Enlace al proyecto:

https://colab.research.google.com/drive/1Jr2DML_SwvcYuWlXMprrOqyBE0eK-8io?usp=sharing

6. Resultados

- **¿Los resultados del cálculo manual coinciden con los de la aplicación? ¿Qué diferencias existen y a qué se deben?**

  Si, los resultados coinciden exactamente, tanto en el desarrollo a mano como en la ejecución en Python, el mensaje plano “HOLA” genero el texto cifrado “NJHl” y fue correctamente recuperado como “HOLA”. La única diferencia radica en la velocidad de cómputo y en la forma en que Python maneja los arreglos multidimensionales frente al desarrollo paso a paso realizado manualmente

- **¿Cuáles fueron las mayores dificultades que encontraste al resolver el problema manualmente? ¿Y al programarlo?**

  En el cálculo manual, la principal dificultad fue aplicar correctamente la reducción de la aritmética modular en números negativos al momento de construir la matriz adjunta (convertir -5 = 21 (mod 26). Al programarlo, el reto consistió en

adaptar el cálculo de la matriz inversa para que funcionara bajo módulo 26, ya que la función genérica numpy.linalg.inv() genera números flotantes decimales no enteros, lo que hizo necesario programar la inversión modular mediante la matriz adjunta y el determinante entero.

- **¿En qué situaciones reales de la computación o ingeniería se aplica el tema que estudiaste?**

  Las transformaciones lineales y la inversión de matrices se aplican en protocolos de seguridad criptográfica, algoritmos de renderizado y gráficos por computadora (rotación y escalado de objetos 3D), procesamiento digital de imágenes y señales, y en la resolución de redes complejas mediante sistemas de ecuaciones lineales.

- **¿Qué aprendiste al construir la aplicación que no hubieras aprendido solo con teoría?**

  Aprendí la importancia de la precisión en los tipos de datos y las limitaciones de las librerías estándar frente a problemas discretos. Entendí que una matriz no solo es una estructura de almacenamiento numérico, sino un operador funcional capaz de transformar la información de manera reversible si y solo si su determinante permite la existencia de una matriz inversa válida.

7. Conclusiones

- **Primera conclusión:** La multiplicación matricial y la inversa de una matriz constituyen la base matemática del Cifrado de Hill, demostrando que la reversibilidad de la encriptación depende directamente de que la matriz clave K posea un determinante cuyo inverso multiplicativo sea válido en el anillo de la aritmética modular Z26
- **Segunda conclusión:** La implementación computacional en Python mediante la librería NumPy facilita el procesamiento de bloques de datos de forma automatizada, reduciendo el margen de error humano en operaciones repetitivas de producto matricial por vectores
- **Tercera conclusión:** La resolución combinada, manual y mediante código permitió verificar

experimentalmente la teoría del álgebra lineal, consolidando el aprendizaje sobre determinantes, matrices adjuntas y vectores aplicados a la seguridad de la información en computación

8. Referencias

[1] R. Larson y D. C. Falvo, *Álgebra Lineal*, 8.ª ed. Ciudad de México, México: Cengage Learning, 2017.

[2] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Boston, MA, USA: Pearson, 2017.

[3] NumPy Developers, "NumPy Reference Guide," *NumPy.org*, 2024. [En línea]. Disponible en: [suspicious link removed]